



MODERN JAVASCRIPT – ASYNC, FETCH, AND FORMS

ABSTRACT

This unit covers essential modern JavaScript features including form validation and processing, client-side storage (Local and Session Storage), async programming with Promises and `async/await`, and the Fetch API for making HTTP requests to external APIs. It focuses on handling user input, managing data, and communicating with servers efficiently in modern web applications.

Unit – 10

Modern Javascript – Async, Fetch, And Forms

Form Processing

Methods to access values of Form elements

Using document.forms[""][].value

```
obj = document.forms["form_name"]["element_name"].value;
```

- Accesses the form using its **name**
- Then accesses the input using its **name**
- Clear and safe when working with multiple forms
- Multiple forms exist on the page

Using form and element names directly(. Method)

```
obj = document.form_name.element_name.value;
```

- Shorter syntax
- Works only if both **form name** and **element name** exist
- When form and elements have **unique name attributes**
- Considered **old-style** and less recommended today

Using getElementById (Best method)

```
obj = document.getElementById("id").value;
```

- Accesses the element using its **id**
- **Most modern and recommended approach**
- Independent of form names

```
<form name= "form_name" action= "#">  
<input type= "text" name= "element_name".....>  
</form>
```

Can access form values by following methods

```
obj = document.forms["form_name"]["element_name"].value;
```

or

```
obj = document.form_name.element_name.value;
```

or

```
obj = document.getElementById(id).value;
```

Note: document.getElementById().value is the most reliable and recommended approach in modern JavaScript.

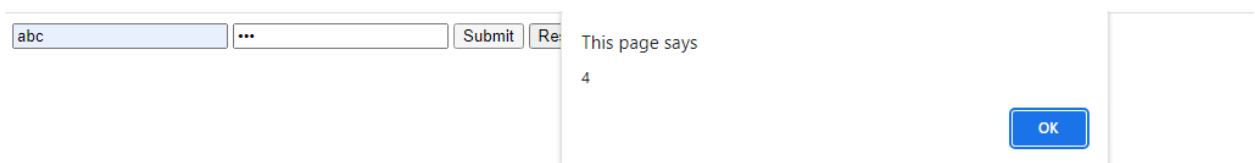
Example : Display User's Entry on Alert using Event (onsubmit)

```
<body>
<form name="f1" onsubmit="return fun()">
  <input type="text" name="t1" id="i1" />
  <input type="submit"/>
</form>
<script>
function fun(){
  // METHOD1
  var obj = document.f1.t1.value;
  // METHOD2
  // var obj = document.forms["f1"]["t1"].value;
  // METHOD3
  // var obj = document.getElementById("i1").value;
  alert(obj);
}
</script>
</body>
```

Example: write a code to find length of form elements.

```
<body>
<form name="f1" onsubmit="fun()">
  <input type="text" name="t1"/>
  <input type="password"/>
  <input type="submit"/>
  <input type="reset"/>
</form>
<script>
function fun()
{
  obj = document.forms["f1"];
  alert(obj.length);
}
</script>
</body>
```

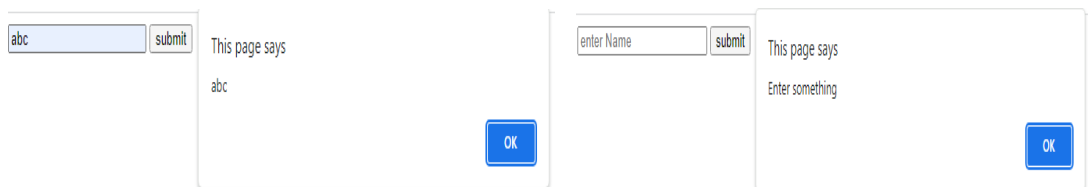
Output: (on clicking submit button.)



Example: Write a code that shows alert which pop up with text “Enter Something” if input field is empty on clicking submit button.

```
<html>
<body>
<form action="#" onsubmit="fun(this)">
  <input type="text" maxlength="10" placeholder="enter Name" id="t1"/>
  <input type="submit" value="submit"/>
</form>
<script type="text/javascript">
function fun(demo)
{
  val=demo.t1.value;
  // OR  val=document.getElementById("t1").value;
  if(val=="")
  {
    alert("Enter something");
    return false;
  }
  alert(val);
}
</script>
</body></html>
```

Output:



Example : Write a JavaScript program to display form elements value; first name, last name, gender, choose game.

```
<html> <head><script>
function f1()
{
  unname=document.form1.uname.value;
  lname=document.form1.lname.value;
  rad=document.form1.r1.value;
  var hob=""
  hobby=document.getElementsByName("ch1");
  for(j=0;j<hobby.length;j++)
  {
```

```

    if(hobby[j].checked)
    {
hob = hob + " " + hobby[j].value ;
    }
    }
    document.write("Uname:"+uname);
    document.write("<br>lname:"+lname);
    document.write("<br>Gender:"+ rad);
    document.write( "<br>Hobby:" + hob );
    }
</script> </head> <body>
<form name="form1" method="get">
FirstName:<input type="text" id="uname" name="uname"/></br>
LastName:<input type="text" id="lname" name="lname"/></br>
Gender: <input type="radio" name="r1" value="male" required/>male
<input type="radio" name="r1" value="female" required/>female </br>
Sports: <input type="checkbox" name="ch1" value="cricket"/> cricket
<input type="checkbox" name="ch1" value="football"/> football
<input type="checkbox" name="ch1" value="basketball"/>basketball </br>
<input type="submit" onclick="f1()">
</form> </body></html>

```

Output:

☐ male ☒ female ☐ cricket ☒ football ☒ basketball

Uname:abc
 lname:abc123
 female
 Hobby: football basketball

Example: Write HTML form accepting an integer having four digits only. Input should not accept characters of letters and special symbols.

```

<html>
<head>
  <script>
    function test(){
      var u=document.getElementById("no").value;
      if(isNaN(u))           //isNaN() method returns true if a value is Not-a-Number.//
    {
      alert("Only digits are allowed")
    }
    else if(u.length != "4")
    {
      alert("Enter four digit no")
    }
  }

```

```
    }  
    else{  
        alert("Perfect")  
    }  
}  
</script>  
</head>  
<body>  
    <input type="text" id="no">  
    <input type="button" onclick="test()" value="submit">  
</body>  
</html>
```

Output:

This page says

Perfect

OK

JavaScript Validation

It is important to validate the form submitted by the user because it can have inappropriate values. So, validation is must to authenticate user.

JavaScript provides facility to validate the form on the client-side so data processing will be faster than server-side validation. Most of the web developers prefer JavaScript form validation.

What is Form Validation in JavaScript?

- ✓ **Form validation** is the process of ensuring that user input in a form is **correct, complete, and in the right format** before it is sent to the server.
- ✓ In **JavaScript**, this is done on the **client-side** (in the browser) to give quick feedback and reduce invalid submissions.
- ✓ The browser itself prevents submission if the field is empty.
But often we want **custom messages or complex rules**, and that's where **JavaScript** comes in.

Through JavaScript, we can validate name, password, email, date, mobile numbers and more fields.

Regular Expression (Regex):

A regular expression is a pattern of characters.

Syntax of ReGex:

/pattern/modifier(s)

test() : This method is called using pattern object and returns true if string is a part of pattern.
syntax:

RegExpObject.test(string)

The String is Required. It is to be searched. It returns true if it finds a match, otherwise false.

Example: String must contain 'abc' anywhere in whole string.

```
<body>
  <input type="text" id="t1"/>
  <input type="submit" onclick="fun()"/>
</script>
function fun(){
  pat = /abc/;
  u = document.getElementById("t1").value;
  if(pat.test(u)){
    document.write("PERFECT"+" &Value is" + u);
```

```
}  
}  
</script>  
</body>
```

Enter Different value in input box:

```
//abc - true  
//abewr = false  
//agrtabc = true  
//ab = false  
//Abc = false
```

Notations to Create R.E.

Modifiers

Modifiers are used to perform case-insensitive and global searches.

Modifier Description

i : Perform case-insensitive matching

Example: Sting contain ABC which is case insensitive

```
<body>  
  <input type="text" id="t1"/>  
  <input type="submit" onclick="fun()"/>  
<script>  
function fun(){  
  pat = /abc/i;  
  u = document.getElementById("t1").value;  
  if(pat.test(u)){  
    alert("PERFECT");  
  }  
}  
</script>  
</body>
```

Output:

```
pat = /abc/i;  
"i" makes string case insensitive. ABC,aBc,ABcxyx11 are allowed.
```


Metacharacters

^	String must start with character For eg. pat= /^A/								
	Allowed	Am	A	Apple					
	Not allowed	conA	Banana	CAT					
*	Matches the preceding character Zero or more times For eg. /bo*/								
	Allowed	b	abcd	boo					
	Not allowed	xyz	oaz						
+	Matches the preceding character one or more times. For eg. /zo+/ matches zoo but not z.								
	Allowed	zo	zoo	Hello zoo					
	Not allowed	Zo	Z	CAT					
?	Matches the preceding character zero or one time. e.g., /[+]?/ matches an optional "+", "-", or an empty string. For eg. /a?ve?/ matches the ve in never.								
	Allowed	v	ve	av					
	Not allowed	a	vv	ae					
\$	Matches with end of particular input For eg. /t\$/								
	Allowed	Cat	Bat	eat					
	Not allowed	Eater	still						
\w	Find the word character i.e. characters from a to z, A to Z, 0 to 9, _								
	Allowed	Apple	_abc	@apple					
	Not allowed	@	@!						
\W	Find a non-word character								
	Allowed	@apple	@	123@					
	Not allowed	0123	Apple	_abc					
\d	Find a digit								
	Allowed	1234	Ab123						
	Not allowed	abcd							
\D	Search non-digit characters i.e all the characters except digits								
	Allowed	Abcd	Ab123						
	Not allowed	1234							
\	Marks the next character as either a special character or a literal. For example, n matches the character n, whereas \n matches a newline character.								
	Noraml Character	.	(	)	\	+	*	?	n
	RegeX	\.	\(	\)	\\	\+	*	\?	\n

Brackets

Brackets are used to find a range of characters

Expressions	Description			
[abc]	Find any (a or b or c= a b c) of the characters inside the brackets			
	Allowed	a12	b	cdef
	Not allowed	C	def	
[^abc]	Find any character, not inside the brackets. (A negative character set.)			
	Allowed	der	de123	abc45
	Not allowed	abc	a	ab
[a-z]	A range of characters. Matches any character in the specified range. For example, [a-z] matches any lowercase alphabetic character in the English alphabet.			
	Allowed	apple	An	Fantastic4
	Not allowed	63	A	\$
[^\m-z]	A negative range of characters. Matches any character that is not in the specified range. For example, [m-z] matches any character that is not in the range m through z.			
	Allowed	apple	123	hi@123
	Not allowed	moon	zoo	test
[0-9]	Find any of the digits between the brackets 0 to 9 (any digit)			
	Allowed	A5	A7	654@
	Not allowed	@	ads	
[^0-9]	Find any digit not in between the brackets (any non-digit)			
	Allowed	Ac3	abc@	12321@
	Not allowed	2131		
(x y)	Find any of the alternatives between x or y separated with Matches either x or y. For example, z wood matches z or wood. (z w)oo matches zoo or wood.			
	Allowed	x	xz	xyz
	Not allowed	X	abc	

For length:

{count}	{n} n is a non-negative integer. Matches exactly n times. For eg /o{2}/ Matches oo (two consecutive o characters).			
	Allowed	foo	book	boook
	Not allowed	Bob	Dog	cat
{min, }	{n,} n is a non-negative integer. Matches the preceding character at least n times. For eg /o{2,}/			
	Allowed	foo	boook	ood
	Not allowed	oaoa	Bat	boat
{min,max}	{n,m} The m and n variables are non-negative integers. Matches the preceding character at least n and at most m times. For eg /o{1,3}/			
	Allowed	Go	Fo	Foo
	Not allowed	Act	Foooo	sky

Example : Must contain only 10 digits.

```

<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
<script>
function fun(){
  pat =/^\\d{10}$/; // \\d means digits. {10} means only 10 characters. ^ means starts with, $
  means ends with.
  u = document.getElementById("t1").value;
  if(pat.test(u)){
    alert("PERFECT");
  }
  else{
    alert("Must contain only 10 digits.")
  }
}
</script>
</body>

```

Examples :
Allowed – 1234567891
Not Allowed: 1234567erd,1234

Example: Must contain only letters and at least one letter or more are allowed.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
  pat = /^[A-Za-z]+$; // [A-Za-z] means letters. + means one or more characters. ^ means
  starts with, $ means ends with. Only letters are allowed.
  u = document.getElementById("t1").value;
  if(pat.test(u))
  {
    alert("PERFECT");
  }
  else{
    alert("Must contain only letters and at least one letter or more are allowed")
  }
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed – abc,xyz,a

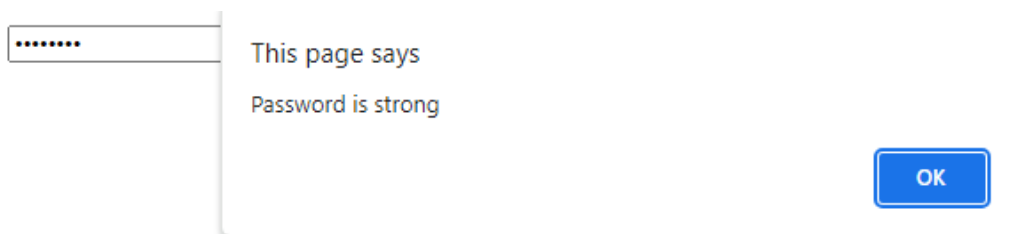
Not Allowed: abc11,1x

Example: Design a login form using JS. Following validation in password field, Minimum length of password must be of 8 letters and it must have some special characters.

```
<body>
  <input type="password" id="p1"/>
  <input type="submit" onclick="fun()"/>
<script type="text/javascript">
function fun(){
  pat=^W/;
  u = document.getElementById("p1").value;
  document.write(u.length);
  if(u.length<8)  {
    alert("Password must be strong...length>=8");
  }
  else if(pat.test(u)==0)  {
    alert("Password must be strong...add one sp.char");
  }
}
```

```
    else {  
        alert("Password is strong");  
    }  
}  
</script>  
</body>  
//////////////////// Using only RegeX method //////////////////////  
<body>  
    <input type="password" id="p1"/>  
    <input type="submit" onclick="fun()"/>  
<script type="text/javascript">  
function fun() {  
    let u = document.getElementById("p1").value;  
    let pat = /^(?=.*\W){8,}$/;  
  
    if (pat.test(u)) {  
        alert("Password is strong");  
    } else {  
        alert("Password must be at least 8 characters and contain a special character");  
    }  
}  
</script>  
</body>
```

Output:



Example: Write a JS to validate username and password. Password: Length must be of 6 to 12 characters. Username: Should not start with _, @ and any number. Both field must not be blank.

```
<html>
<head>
<script type="text/javascript">
function fun() {

    let u = document.getElementById("t1").value;
    let v = document.getElementById("p1").value;

    let userPattern = /^[^_@\\d].*$/;
    let passPattern = /^[^]{6,12}$/;

    if (!userPattern.test(u)) {
        alert("Invalid username (cannot start with _, @ or digit)");
    }
    else if (!passPattern.test(v)) {
        alert("Password must be 6 to 12 characters");
    }
    else {
        alert("Username and Password are valid");
    }
}
</script>
</head>

<body>
NAME <input type="text" id="t1"/><br><br>
PASS <input type="password" id="p1"/><br><br>
<input type="submit" value="Login" onclick="fun()"/>
</body>
</html>
```

Output:

NAME PASS

This page says
Password is strong

OK

Example: Show validation using JS on fields like name, phone number and email id

RE for Username: /^[A-z]+\$;

RE for Phone number: /\d{10}\$/;

RE for email: /\w+([\.-]?\w+)*@(\w+([\.-]?\w+)*(\.\w{2,3})+)\$/

1. The two forward-slashes `/.../` contains a regex.
2. The leading `^` and trailing `$` match the beginning and the ending of the input string, respectively. That is, the entire input string shall match with this regex, instead of a part of the input string.
3. `\w+` matches 1 or more word characters (a-z, A-Z, 0-9 and underscore).
4. `[\.-]` matches character `.` or `-`. We need to use `.` to represent `.` as `.` has special meaning in regex. The `\` is known as the escape code, which restore the original literal meaning of the following character.
5. `[\.-]?` matches 0 or 1 occurrence of `[\.-]`.
6. Again, `\w+` matches 1 or more word characters.
7. `([\.-]?\w+)*` matches 0 or more occurrences of `[\.-]?\w+`.
8. The sub-expression `\w+([\.-]?\w+)*` is used to match the username in the email, before the `@` sign. It begins with at least one word character (a-z, A-Z, 0-9 and underscore), followed by more word characters or `.` or `-`. However, a `.` or `-` must follow by a word character (a-z, A-Z, 0-9 and underscore). That is, the string cannot contain `".."`, `"--"`, `".-"` or `"-."`. Example of valid string are `"a.1-2-3"`
9. The `@` matches itself
10. Again, the sub-expression `\w+([\.-]?\w+)*` is used to match the email domain name, with the same pattern as the username described above.
11. The sub-expression `\.\w{2,3}` matches a `.` followed by two or three word characters, e.g., `".com"`, `".edu"`, `".us"`, `".uk"`, `".co"`
12. `(\.\w{2,3})+` specifies that the above sub-expression shall occur one or more times, e.g., `".com"`, `".co.uk"`, `".edu.sg"` etc.

Example: Design a login form using JS. Following validation in password field, Minimum length of password must be of 8 letters and it must have some special characters.

```
<html>
<head>
<script type="text/javascript">
function fun(){
    pat=/W+;
    u = document.getElementById("p1").value;
    if(u.length <= 8){
        alert("Password must be greater than 8");
    }
    else if(!pat.test(u))
    {
        alert("Password must be strong...Add at least one special char");
    }
    else{
        alert("Password Entered: " + u);
    }
}
</script>
</head>
<body>
<input type="password" id="p1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```


Email Regex

Sample Pattern: `/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/`

Example: `abc.test@yahoo.co.in`

Regex Part	Meaning	Example
<code>^</code>	Start of email	—
<code>\w+</code>	Name (letters/numbers/_)	rahul, amit123
<code>([.-]?\w+)*</code>	Optional . or - in name	rahul.kumar, amit-singh
<code>@</code>	Must have @	@
<code>\w+</code>	Domain name	gmail, yahoo, gov
<code>([.]?\w+)*</code>	Subdomain (optional)	mail.nic
<code>(\.\w{2,3})+</code>	Domain extension	.com, .in, .co.in
<code>\$</code>	End of email	

Valid Emails

akshay.kumar@gmail.com
 amit-shah@yahoo.co.in
 kat123@gov.in
 info@mail.nic.in

Invalid Emails

.psp@gmail.com ✗ starts with dot
 psp@gmail ✗ no domain extension
 psp@gmail.india ✗ extension too long
 psp@@gmail.com ✗ double @

Example:

```
<body>
<form>
  <label>Email:</label><br><br>
  <input type="text" id="email" placeholder="rahul.kumar@gmail.com">
  <br><br>
  <button type="button" onclick="validateEmail()">Submit</button>
</form>
<p id="result"></p>
<script>
function validateEmail() {
  const email = document.getElementById("email").value;
  const regex = /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/;
  document.getElementById("result").innerText =
    regex.test(email) ? "Valid Email" : "Invalid Email";
}
</script>
</body>
```

Another Simple email

Example: Email Address `/^\w+@\w+ (\.\w{2,3})+$/`

Example: lju@gmail.com

<code>/^[0-9]\$/</code> - This regex ensures that the input is exactly one digit , with no extra characters before or after.

^: Matches the start of the string. [0-9]: Matches any single digit from 0 to 9. \$: Matches the end of the string. Matches: 0, 5, 9.
--

<code>/^[A-Z]\$/</code> - This regex ensures that the input contains exactly one uppercase letter , and nothing else.
--

^: Anchors the regex to the start of the string. [A-Z]: Matches any single uppercase English letter (from A to Z). \$: Anchors the regex to the end of the string. Matches: A, Z, M.
--

}

Regex Cheat sheet

Regex	Description	Matches (Examples)	Does Not Match
/abc/	Must contain substring abc anywhere	abc, xabc, 123abcxyz	ab, xyz
/abc/i	Case-insensitive match for abc	ABC, aBc, ABcxyz	xyz
/[abc]/	Must contain at least one of a, b, or c	a, cab, xyzab	xyz, ddf
/[^abc]/	Must contain at least one character other than a, b, or c	cat, the, 123d	a, abc, cbbca
/[0-9]/	Must contain at least one digit	a1, 123, the2	abc, @xyz
/[^\d]/	Must contain at least one non-digit	a1, @12, xyz9	1, 123, 999
/(x y)/	Must contain x or y	x, y, x1y2, asdx, y123	11, zzz
/^A/	Must start with A	Apple, A1, Awesome	a123, bA
/Bo*/	Must contain B followed by zero or more o	B, Bo, Boo, Born	bo, table
/t\$/	Must end with t	Bat, Best, 123t	top, table1
/^\d{10}\$/	Exactly 10 digits	1234567890	1234, abc1234567
/^[A-Za-z]+\$/	Only letters (uppercase/lowercase), at least one required	abc, XYZ, a	abc123, 1a, @b
/^\d{4}\$/	Exactly 4 digits	1234, 9876	123, 12a4
/^[_@0-9]/	Username must start with _, @, or a digit	_abc, @user, 1name	user1, abc_1, hello123
/\W+/	Must contain at least one special character	abc@123, pass!word	password123, abcde
/^\w+([.-]?*\w+)*@\w+([.]?*\w+)*(\.\w{2,3})+\$/	Email validation	test@gmail.com, abc.xy@domain.co.in	abc@com, @gmail.com
/^[0-9]\$/	Exactly one digit (0–9)	0, 5, 9	12, a, 99
/^[A-Z]\$/	Exactly one uppercase letter	A, Z, M	AB, a, 1
/[+-]?/	Optional + or –	+5, -5, 5	
/^[^xy]\$/	One character, not x or y	a	x, y
/^[^xy]+\$/	One or more chars, not x or y	abc	axc
/^[^xy]{3}\$/	Exactly 3 chars, not x or y	abc, 123, ab_	ab, abcd

Local Storage and Session Storage

What is Web Storage?

JavaScript provides **Web Storage API** for saving key-value data directly in the browser. Two main types:

- **localStorage** → Data stored permanently (until manually cleared).
- **sessionStorage** → Data stored only for the current browser tab/session.

 Storage Comparison

Feature	Local Storage	Session Storage
Capacity	~10 MB	~5 MB
Browsers	HTML 5	HTML 5
Accessible From	Any window	Same tab
Expiration	Never	On tab close
Browser Support	Very high	Very high
Supported Data Types	String only	String only
Auto-expire Option	No	Yes
Storage Location	Browser only	Browser only
Sent with Requests	No	No
Editable / Blockable	Yes	Yes

Local Storage

- Persists even after the browser is closed/reopened.
- Good for saving **preferences, settings, theme, cached data**.

Example:

```
// Save
localStorage.setItem("username", "ABC");

// Retrieve
let user = localStorage.getItem("username");
console.log(user); // "ABC"

// Update
localStorage.setItem("username", "ABC");

// Delete one item
localStorage.removeItem("username");

// Clear all
localStorage.clear();
```

Session Storage

- Data lasts **only while the browser tab is open**.
- Once the tab is closed, data is gone.
- Good for **temporary data** like form input, step-by-step processes.

Example:

```
// Save
sessionStorage.setItem("cart", "3 items");

// Retrieve
let cart = sessionStorage.getItem("cart");
console.log(cart); // "3 items"

// Remove
sessionStorage.removeItem("cart");

// Clear all
sessionStorage.clear();
```

Method	Description	Syntax	Example
setItem(key, value)	Stores a key–value pair. Value is always saved as a string .	storage.setItem("key", "value")	localStorage.setItem("username", "Alice");
getItem(key)	Retrieves the value for the given key. Returns null if not found.	storage.getItem("key")	localStorage.getItem("username"); // "Alice"
removeItem(key)	Removes a specific item from storage (only that key).	storage.removeItem("key")	localStorage.removeItem("username");
clear()	Removes all items from storage.	storage.clear()	localStorage.clear();

Steps to Observe

Step 1: Open your HTML page

- Open your HTML file in the browser.

Step 2: Open Developer Tools

- **Right-click anywhere on the page**
- Click **Inspect**
- **OR**
- Press **F12**
- Or **Ctrl + Shift + I** (Windows)
- Or **Cmd + Option + I** (Mac)

Step 3: Go to the Application tab

- In Developer Tools, click **Application**
- If not visible, click the » **(double arrow)** and select **Application**

Step 4: Find Storage section (Left panel)

Under **Storage**, you will see:

- **Local Storage**
- **Session Storage**

Step 5: Click your website

- Click the URL shown under:
 - **Local Storage** → to see localStorage
 - **Session Storage** → to see sessionStorage

Example: Login Demo (Save/Show/Clear)

```
<!DOCTYPE html>
<html>
<body>
  <h2>Login Demo</h2>
  <input type="text" id="username" placeholder="Enter username">
  <button onclick="saveUser()">Save</button>
  <button onclick="showUser()">Show</button>
  <button onclick="clearUser()">Clear</button>

  <p id="output"></p>

  <script>
    function saveUser() {
      let user = document.getElementById("username").value;
      localStorage.setItem("username", user); // persists
      sessionStorage.setItem("username", user); // temp
    }

    function showUser() {
      let localUser = localStorage.getItem("username");
      let sessionUser = sessionStorage.getItem("username");
      document.getElementById("output").innerHTML =
        "Local: " + localUser + "<br>Session: " + sessionUser;
    }

    function clearUser() {
      localStorage.clear();
      sessionStorage.clear();
      document.getElementById("output").innerHTML = "Cleared!";
    }
  </script>
</body>
</html>
```

Login Demo

By Clicking on **save** button

The screenshot shows the Chrome DevTools Application tab. The left sidebar shows the storage hierarchy: Local storage, Session storage, and IndexedDB. Under Local storage, the URL 'http://127.0.0.1:5500' is selected. The main pane shows a table with one key-value pair:

Key	Value
username	abc

By Clicking on **show** button

Login Demo

Local: abc
Session: abc

By Clicking on **clear** button

The screenshot shows the Chrome DevTools Application tab. The left sidebar shows the storage hierarchy. Under Local storage, the URL 'http://127.0.0.1:5500' is selected. The main pane shows an empty table:

Key	Value
-----	-------

Example:

Create a small HTML page where user store two fields in sessionStorage:

```
username = "ab"
```

```
email = "ab@example.com"
```

Retrieve and display the stored username in the console.

Remove only the email field from sessionStorage, then print the remaining data in the console.

```
<script>
  // Store data in sessionStorage
  sessionStorage.setItem("username", "lj");
  sessionStorage.setItem("email", "lj@example.com");

  // Retrieve and display username
  console.log("Username:", sessionStorage.getItem("username"));

  // Remove only the email field
  sessionStorage.removeItem("email");

  // Print remaining data
  console.log("Remaining sessionStorage data:");
  console.log("Username:", sessionStorage.getItem("username"));
  console.log("Email:", sessionStorage.getItem("email")); // null
</script>
```

Example:

Create a small HTML page where user store three fields in localStorage:

```
product = "Laptop"
```

```
price = 50000
```

```
discount = 10
```

Retrieve and display the stored product in the console.

Remove the discount field from localStorage, then print the updated data in the console.

```
<script>
  // 1. Store data in localStorage
  localStorage.setItem("product", "Laptop");
  localStorage.setItem("price", 50000);
  localStorage.setItem("discount", 10);

  // 2. Retrieve and display product
  console.log("Product:", localStorage.getItem("product"));

  // 3. Remove discount field
  localStorage.removeItem("discount");

  // 4. Print updated data
  console.log("Updated localStorage data:");
  console.log("Product:", localStorage.getItem("product"));
  console.log("Price:", localStorage.getItem("price"));
  console.log("Discount:", localStorage.getItem("discount")); // null
</script>
```


Example:

Create a form with three fields: product, price, discount and a Save button.

Requirements:

1. When the user clicks Save, store the entered values into localStorage.
2. After saving, display a message: Data saved in local storage!
3. Add another button Remove Discount that removes only the discount field from localStorage.
4. After removal, display the remaining data in the console.

```
<body>
<h3>Product Details</h3>
<form onsubmit="return saveData()">
  Product: <input type="text" id="product" required><br><br>

  Price: <input type="number" id="price" required><br><br>

  Discount: <input type="number" id="discount" required><br><br>

  <input type="submit" value="Save">
</form>

<button onclick="removeDiscount()">Remove Discount</button>

<script>
function saveData() {
  // Store values in localStorage
  localStorage.setItem("product", document.getElementById("product").value);
  localStorage.setItem("price", document.getElementById("price").value);
  localStorage.setItem("discount", document.getElementById("discount").value);

  // Display message
  alert("Data saved in local storage!");

  return false; // prevent page refresh
}

function removeDiscount() {
  // Remove only discount
  localStorage.removeItem("discount");

  // Display remaining data in console
  console.log("Remaining localStorage data:");
  console.log("Product:", localStorage.getItem("product"));
  console.log("Price:", localStorage.getItem("price"));
  console.log("Discount:", localStorage.getItem("discount")); // null
}
</script>
</body>
```

Example (* Ref only) – Store Multiple data

```
<body>
<h2>Login Demo</h2>

<input type="text" id="username" placeholder="Enter username">
<button onclick="saveUser()">Save</button>
<button onclick="showUser()">Show</button>
<button onclick="clearUser()">Clear</button>

<p id="output"></p>

<script>
function saveUser() {
  let user = document.getElementById("username").value;

  // get old data or empty array
  let users = JSON.parse(localStorage.getItem("users")) || [];

  // add new value
  users.push(user);

  // save again
  localStorage.setItem("users", JSON.stringify(users));
}

function showUser() {
  let users = JSON.parse(localStorage.getItem("users")) || [];
  document.getElementById("output").innerHTML =
  "Users:" + users.join("<br>");
}

function clearUser() {
  localStorage.clear();
  document.getElementById("output").innerHTML = "Cleared!";
}
</script>

</body>
</html>
```

Asynchronous JavaScript

JavaScript normally executes code **synchronously**, meaning one line runs after another. However, some operations such as **API calls, timers, file loading, and database access** take time to complete.

Asynchronous JavaScript allows these time-consuming operations to run in the background **without blocking the rest of the program**.

Asynchronous Programming is Needed

Without asynchronous handling:

- The browser would freeze while waiting
- User experience would be poor
- Long tasks would block UI interaction

Asynchronous programming ensures:

- Better performance
- Responsive applications
- Non-blocking execution

Method -1 Promises

A **Promise** is an object that represents the **future result** of an asynchronous operation.

Promises are objects that represent the eventual completion (or failure) of an asynchronous operation and its resulting value. They provide a cleaner way to handle async operations than callbacks, using a chain of methods.

A Promise can be in one of three states:

1. **Pending** – operation is still running
2. **Fulfilled** – operation completed successfully
3. **Rejected** – operation failed

Promise Creation Syntax

```
let promise = new Promise((resolve, reject) => {  
  // asynchronous operation  
});
```

- `resolve(value)` → called when the task is successful
- `reject(error)` → called when the task fails

Promise Handling Methods

`.then()` – Success Handler

Runs when the Promise is **fulfilled**.

`.catch()` – Error Handler

Runs when the Promise is **rejected**.

`.finally()` – Always Executes whether the promise is fulfilled or rejected.

Runs **regardless of success or failure**

Syntax:

```
let promise = new Promise((resolve, reject) => {  
  // async task  
  if (success) {  
    resolve(result);  
  } else {  
    reject(error);  
  }  
});  
promise  
  .then(result => console.log(result))  
  .catch(error => console.log(error))  
  .finally(() => console.log("Process finished"));
```

How it works:

A **Promise** in JavaScript is used to handle asynchronous operations such as fetching data from a server, reading files, or executing delayed tasks.

The syntax `let promise = new Promise((resolve, reject) => { ... })` creates a new Promise object that represents a task whose result will be available in the future.

When `new Promise()` is executed, JavaScript immediately runs the **executor function** provided inside it. This executor function automatically receives two parameters: `resolve` and `reject`. These are special callback functions provided by JavaScript itself and are used to control the final outcome of the promise.

The `resolve(value)` function is called when the asynchronous operation completes successfully. Calling `resolve()` changes the promise state from **pending** to **fulfilled**, and the value passed to `resolve()` becomes available to the `.then()` method for further processing.

The `reject(error)` function is called when the asynchronous operation fails or encounters an error. Calling `reject()` changes the promise state from **pending** to **rejected**, and the error passed to `reject()` is received by the `.catch()` method, allowing error handling in a clean and structured way.

Initially, every promise starts in the **pending** state. Based on the result of the asynchronous task, the promise is settled either by calling `resolve()` (success) or `reject()` (failure). Once settled, the promise state cannot be changed again.

Purpose of setTimeout()

setTimeout() is used to:

- Delay execution of code
- Simulate real-time delays (API calls, loading, waiting)

Syntax

```
setTimeout(function, delayInMilliseconds);
```

Example

```
setTimeout(() => {  
  console.log("Executed after 2 seconds");  
}, 2000);
```

When to Use setTimeout()

Use setTimeout() when:

- Simulating asynchronous behavior
- Delaying execution
- Showing loaders or messages after a delay
- Testing Promise behavior

Complete Promise Example using setTimeout()

```
let order = new Promise((resolve, reject) => {  
  let success = true;  
  
  setTimeout(() => {  
    if (success) {  
      resolve("Order completed");  
    } else {  
      reject("Order failed");  
    }  
  }, 2000);  
});  
  
order  
  .then(resolve => console.log(resolve))  
  .catch(reject => console.log(reject))  
  .finally(() => console.log("Process finished"));
```

Method -2 Async / Await

async / await is a **modern way to write asynchronous JavaScript**.

It is built **on top of Promises** and makes asynchronous code look like **normal synchronous code**.

Why async / await is Used

Promises using `.then()` and `.catch()` work well, but:

- They can become **hard to read**
- Chained `.then()` calls look messy

async / await solves this by:

- Making code **cleaner**
- Making logic **easier to understand**
- Improving **error handling**

async Keyword

Purpose

The `async` keyword is used **before a function** to indicate that:

- The function will work with asynchronous code
- The function **always returns a Promise**

Syntax

```
async function functionName() {  
  // async code  
}
```

await Keyword

Purpose

The `await` keyword is used to:

- Pause execution until a Promise is resolved
- Retrieve the resolved value of the Promise

`await` can be used **only inside an async function**.

Syntax

```
let result = await promise;
```

Promise in async/await

```
return new Promise((resolve,reject) => {  
  // async work  
});
```

It means:“This function will not give the result immediately. It promises to give the result later.”

```
new Promise(...)
```

Creates a Promise object

A Promise represents a future value

async/await works only with Promises, not with callbacks. To use async/await, convert callback-based code into a Promise.

What is try...catch in async/await?

When using async/await, **errors are handled using try...catch** instead of .catch().

- try → code that may succeed or fail
- catch → runs if an error occurs

Syntax

```
async function functionName() {
  try {
    let result = await someAsyncFunction();
    // success code
  } catch (error) {
    // error handling
  }
}
```

Example

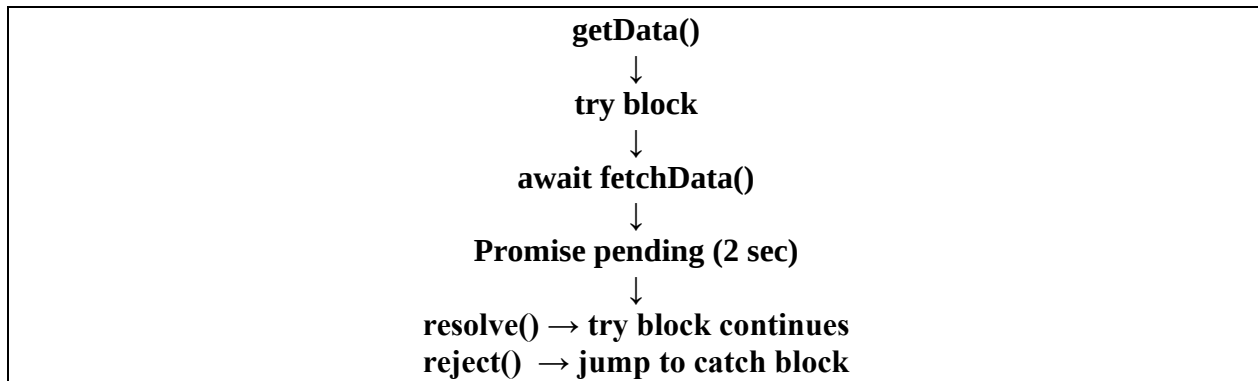
```
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let success = true; // change to false to test error

      if (success) {
        resolve("Data received");
      } else {
        reject("Error while fetching data");
      }
    }, 2000);
  });
}

async function getData() {
  try {
    let result = await fetchData(); // wait for Promise
    console.log(result);           // success
  } catch (error) {
    console.log("Error:", error); // error handling
  }
}

getData();
```

How it works



The function `fetchData()` is responsible for creating and returning a **Promise**. Inside this function, `new Promise((resolve, reject) => { ... })` is used to simulate an asynchronous task using `setTimeout()`. The promise remains in a **pending** state for 2 seconds. After the delay, a variable `success` is checked to decide the outcome of the operation.

If success is true, the promise is **resolved** using `resolve("Data received")`. This indicates that the asynchronous task has completed successfully. If success is false, the promise is **rejected** using `reject("Error while fetching data")`, indicating a failure during the operation.

The function `getData()` is declared using the `async` keyword. Declaring a function as `async` allows the use of the `await` keyword inside it, making asynchronous code look and behave like synchronous code.

Inside the `try` block, the statement `await fetchData()` is executed. At this point, JavaScript **pauses the execution** of the `getData()` function until the promise returned by `fetchData()` is settled. While waiting, the promise stays in the **pending** state for 2 seconds.

If the promise is **resolved**, the resolved value ("Data received") is returned by `await fetchData()` and assigned to the variable `result`. The execution then continues inside the `try` block, and the success message is printed using `console.log(result)`.

If the promise is **rejected**, the error is automatically **thrown** by `await`. Control immediately jumps from the `try` block to the `catch` block, where the error is received and printed using `console.log("Error:", error)`.

Finally, when `getData()` is called, the output depends on the value of `success`. If success is true, the output is **"Data received"**. If success is false, the output is **"Error: Error while fetching data"**. This demonstrates how `async/await` works with `try-catch` to handle asynchronous operations in a clear and structured manner.

Examples :

Write a function that returns a Promise which resolves a number after 1 second. Use a loop with async/await to get values from 1 to 3. Print the doubled result of each value.

```
<!DOCTYPE html>
<html>
<body>

<script>
/* Function that returns a Promise
   Resolves the given number after 1 second */
function getNumber(num) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(num);
    }, 1000);
  });
}

/* Async function using loop and await */
async function processNumbers() {
  for (let i = 1; i <= 3; i++) {
    let value = await getNumber(i); // wait for promise
    console.log(value * 2);         // print doubled value
  }
}

processNumbers();
</script>
</body>
</html>
```

- `getNumber(num)`
 - returns a Promise
 - resolves the number after **1 second**
- `processNumbers()`
 - uses `async/await`
 - loop runs from **1 to 3**
 - waits 1 second for each number
 - prints the **doubled value**

Example :2

Prompt the user to enter 3 numbers (you can use `prompt()` in browser). For each number, return a Promise that resolves after 1 second. Print the triple of each entered number.

```
<body>
<script>
/* Function that returns a Promise
   Resolves the number after 1 second */
function processNumber(num) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(num * 3); // triple the number
    }, 1000);
  });
}
/* Async function to handle user input */
async function getUserNumbers() {
  for (let i = 1; i <= 3; i++) {
    let input = prompt("Enter number " + i + ":");
    let number = Number(input);

    let result = await processNumber(number); // wait 1 second
    console.log("Triple:", result);
  }
}
getUserNumbers();
</script>
</body>
```

- User is prompted **3 times** to enter numbers
- Each number is passed to a Promise
- Promise resolves **after 1 second**
- The **triple of each number** is printed in the console

Fetch API

The **Fetch API** is a modern JavaScript interface used to **send HTTP requests** and **receive responses** from external APIs or servers.

It works asynchronously and returns a **Promise**.

HTTP GET Request using Fetch API

A **GET request** is used to **retrieve data** from a server.

Syntax

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

Example: GET Request with Fetch API

```
fetch("https://dog.ceo/api/breeds/image/random")
  .then(response => response.json())
  .then(data => {
    console.log("Image URL:", data.message);
    console.log("Status:", data.status);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

Concept used:

- `fetch(url)` → Sends HTTP GET request
- `response` → Server response object
- `.json()` → Converts JSON response to JavaScript object
- `.then()` → Handles successful response
- `.catch()` → Handles errors

Error Handling in Fetch API

Fetch **does not automatically fail** on HTTP errors (404, 500).

You must check the response status manually using `response.ok` concept.

- `response.ok` → Check HTTP status

Example with Proper Error Handling

```
fetch("https://dog.ceo/api/breeds/image/random")
  .then(response => {
    if (!response.ok) {
      throw new Error("Failed to fetch data");
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Error:", error.message);
  });
```

Example -1

Create a simple web page that allows users to fetch and display a random dog image from an external API. The page should include:

1. A heading that indicates the purpose of the page (e.g., "Random Dog Image").
2. A button that, when clicked, sends a request to the Dog CEO API (<https://dog.ceo/api/breeds/image/random>) to fetch a random dog image.
3. An image element that displays the fetched dog image dynamically when the user clicks the button.
4. The solution should use JavaScript's fetch API and async/await syntax to handle the API request.

```
<body>
  <h2>Random Dog Image</h2>
  <button onclick="getDog()">Get Dog</button>
  <img id="dogImg" width="300">
  <script>
    function getDog() {
      fetch("https://dog.ceo/api/breeds/image/random")
        .then(response => response.json())
        .then(data => {
          document.getElementById("dogImg").setAttribute("src", data.message);
          console.log("Status:", data.status);
        })
        .catch(error => {
          console.log("Error:", error);
        });
    }
  </script>
</body>
```

- Displays a **Random Dog Image**
- When the user clicks the **Get Dog** button
- It fetches a random dog image from an **external API**
- Shows the image on the page

Example -2 : Fetch Random joke API and display on click

```
<body>
<h2>Random Joke</h2>
<button onclick="getJoke()">Get Joke</button>
<p id="joke"></p>

<script>
function getJoke() {
  fetch("https://official-joke-api.appspot.com/random_joke")
    .then(res => res.json())
    .then(data => {
      document.getElementById("joke").innerHTML =
        data.setup + "<br><b>" + data.punchline + "</b>";
    });
}
</script>

</body>
```